



OMNICART MARKETPLACE PROGRAM

Low Level Design Document

Service-level design, schemas, payloads, sequence flows, validations, state machine, error handling, and test hooks for payment gateway integrations.

Project:	OmniCart Commerce Platform with Multi-Gateway Payments
Prepared For:	Product, Engineering, QA, Security, Operations, Finance
Generated:	2026-05-13
Classification:	Synthetic test data for ingestion pipeline validation
Version:	1.0

1. Payment Orchestrator Module Design

The payment orchestrator exposes internal APIs used by Checkout and external callback APIs used by payment gateways. It owns the payment state machine, attempt ledger, routing decisions, gateway adapters, idempotency, and reconciliation events.

2. Package Structure

```
com.omnicart.payments
  api/PaymentIntentController.java
  api/GatewayWebhookController.java
  domain/PaymentIntent.java
  domain/PaymentAttempt.java
  domain/PaymentStateMachine.java
  routing/GatewayRoutingEngine.java
  adapter/PaymentGatewayAdapter.java
  adapter/razorpay/RazorpayAdapter.java
  adapter/stripe/StripeAdapter.java
  ledger/PaymentLedgerService.java
  webhook/WebhookSignatureVerifier.java
  reconciliation/GatewaySettlementImporter.java
```

3. Database DDL Contracts

```

create table payment_intents (
  id varchar(32) primary key,
  order_intent_id varchar(32) not null unique,
  customer_id varchar(32) not null,
  amount numeric(12,2) not null check (amount >= 0),
  currency char(3) not null,
  status varchar(32) not null,
  capture_policy varchar(32) not null,
  idempotency_key varchar(128) not null unique,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table payment_attempts (
  id varchar(32) primary key,
  payment_intent_id varchar(32) references payment_intents(id),
  attempt_no int not null,
  gateway_code varchar(32) not null,
  gateway_transaction_id varchar(128),
  status varchar(32) not null,
  failure_code varchar(64),
  failure_message text,
  raw_gateway_status varchar(64),
  created_at timestamptz not null default now(),
  unique(payment_intent_id, attempt_no),
  unique(gateway_code, gateway_transaction_id)
);

create table gateway_webhook_events (
  id uuid primary key default gen_random_uuid(),
  gateway_code varchar(32) not null,
  event_id varchar(128) not null,
  event_type varchar(64) not null,
  signature_valid boolean not null,
  payload jsonb not null,
  processed_at timestamptz,
  unique(gateway_code, event_id)
);

```

4. Sequence Flow: Card Authorization

```

Storefront -> Checkout: placeOrder(cartId, addressId, paymentSelection)
Checkout -> Payment Orchestrator: createIntent(orderIntent)
Payment Orchestrator -> Routing Engine: resolveRoute(context)
Payment Orchestrator -> Fraud Service: score(paymentContext)
Payment Orchestrator -> RazorpayAdapter: authorize(payload)
RazorpayAdapter -> Gateway: /payments/authorize
Gateway --> RazorpayAdapter: authorized or action_required
Payment Orchestrator -> Ledger: append AUTH_HOLD entry
Payment Orchestrator -> Kafka: payment.authorized
Order Service -> Customer Notification: order confirmed

```

5. Validation Rules

Validation	Rule	Error
Amount	Sum of tender lines must equal order payable amount.	PAYMENT_AMOUNT_MISMATCH
Currency	Currency must match marketplace country configuration.	UNSUPPORTED_CURRENCY
Idempotency	Same key and payload hash returns existing response.	IDEMPOTENCY_PAYLOAD_CONFLICT
Gateway	Selected gateway must support tender type and merchant account.	GATEWAY_NOT_ELIGIBLE
Refund	Refund amount cannot exceed captured remaining amount.	REFUND_AMOUNT_EXCEEDED

6. Error Mapping

Gateway Error	Canonical Code	User Message	Retry?
insufficient_funds	PAYMENT_DECLINED_FUNDS	Payment was declined. Try another method.	No same-card retry
issuer_unavailable	ISSUER_TEMPORARILY_UNAVAILABLE	Your bank is temporarily unavailable.	Yes alternate route
timeout	GATEWAY_TIMEOUT	We are checking your payment status.	Status inquiry first
invalid_signature	WEBHOOK_SIGNATURE_INVALID	Not shown to customer	No

7. Detailed API Schema: Refund

```
POST /api/v1/refunds
Request:
{
  "orderId": "ord_10091",
  "lines": [
    {"orderLineId": "ol_1", "amount": 799.00, "reason": "CUSTOMER_RETURN"}
  ],
  "requestedBy": "support_agent_21",
  "idempotencyKey": "refund_ord_10091_ol_1_001"
}
Response:
{
  "refundId": "rf_00991",
  "status": "REFUND_PENDING",
  "estimatedCompletion": "2026-05-16",
  "tenderBreakdown": [
    {"type": "CARD", "amount": 699.00},
    {"type": "GIFT_CARD", "amount": 100.00}
  ]
}
```

8. Test Hooks and Observability Fields

- Header `X-Test-Gateway-Scenario` can be used in lower environments to simulate gateway timeout, decline, `action_required`, duplicate webhook, and delayed webhook.
- Every API response includes `correlationId`.
- Payment attempt stores `gateway_latency_ms`, `route_id`, `risk_score`, and `idempotency_key`.

- Contract tests use OpenAPI examples and gateway sandbox mocks.

9. OpenAPI Excerpt

```
PaymentIntent:
  type: object
  required: [paymentIntentId, status, amount, currency]
  properties:
    paymentIntentId: { type: string, example: "pi_781120" }
    status: { type: string, enum: [CREATED, AUTHORIZING, ACTION_REQUIRED, AUTHORIZED, CAPTURED, FAILED, REFUNDED] }
  }
  amount: { type: number, format: decimal }
  currency: { type: string, minLength: 3, maxLength: 3 }
  attempts:
    type: array
    items:
      $ref: '#/components/schemas/PaymentAttempt'
```